

Implementação de uma Rede P2P para Simulação de DTN em Cenários de Alta Latência e Desconexão

Universidade Federal do Amazonas
Disciplina de Sistemas Distribuídos

Márcio Éric Lamêgo Valente

José Santo de Moura Neto

25 de maio de 2026

Sumário

1	Introdução	3
2	Objetivos	3
3	Arquitetura da Solução	3
4	Modelo de Contatos	3
5	Roteamento Epidêmico	4
6	Interface de Execução	5
7	Cenários de Teste	5
8	Métricas	6
9	Resultados Obtidos	6
10	Testes	6
11	Extensão PROPHET	6
12	Conclusão	7

1 Introdução

Este relatório apresenta a implementação de uma rede P2P simulada para cenários de DTN (*Delay-Tolerant Network*), conforme a atividade prática da disciplina de Sistemas Distribuídos. O objetivo principal foi demonstrar, por meio de simulação, como mensagens podem ser armazenadas e encaminhadas em uma rede sem conectividade fim-a-fim garantida.

Em redes DTN, os nós podem ficar desconectados durante longos períodos. Por isso, cada nó atua como uma entidade autônoma capaz de armazenar mensagens localmente e encaminhá-las quando encontra outro nó. Esse modelo é conhecido como *store-and-forward*.

2 Objetivos

Os objetivos da implementação foram:

- simular uma arquitetura P2P sem servidor central;
- representar cada nó com identificador, buffer de mensagens, relógio lógico e lista de mensagens vistas;
- implementar roteamento epidêmico para encaminhamento de mensagens;
- simular contatos intermitentes entre pares de nós;
- avaliar taxa de entrega, latência média e overhead;
- executar testes em cenários denso, esparso e em cadeia.

3 Arquitetura da Solução

A solução foi implementada em Python e utiliza uma simulação baseada em eventos discretos. Cada nó P2P é representado por uma entidade independente dentro do simulador. O simulador não funciona como servidor central de mensagens; ele apenas controla o avanço do tempo e agenda os eventos de contato definidos nos arquivos de entrada.

Os principais módulos do sistema são:

4 Modelo de Contatos

Os contatos são definidos em arquivos `contacts.txt`. Cada linha indica o intervalo em que dois nós podem se comunicar:

Módulo	Responsabilidade
SimulationEngine	Carrega entradas, agenda eventos e coordena a simulação.
ContactParser	Lê o arquivo de contatos no formato <code>tempo_inicio, tempo_fim, noA, noB</code> .
MessageParser	Lê mensagens iniciais e gera identificadores de mensagens.
Node	Representa um nó P2P com buffer, mensagens vistas, relógio lógico e probabilidades de entrega.
EpidemicRouter	Implementa a troca de resumos e transferência de mensagens ausentes.
MetricsCollector	Registra entregas, transmissões, latência e overhead.
CLI	Disponibiliza comandos para execução da simulação.

```
tempo_inicio, tempo_fim, noA, noB
```

Exemplo:

```
0.0, 5.0, 1, 2
5.1, 8.0, 2, 3
8.2, 12.0, 3, 1
```

Durante a simulação, os eventos são processados em ordem crescente de tempo. Quando ocorre um contato, os dois nós envolvidos executam a troca P2P.

5 Roteamento Epidêmico

O roteamento epidêmico foi implementado com base na troca de resumos de mensagens. Quando dois nós entram em contato, cada um informa ao outro quais mensagens já conhece. Em seguida, cada nó envia as mensagens que possui e que o outro ainda não viu.

O processo executado em cada contato é:

1. atualizar o relógio lógico dos nós para o tempo do contato;
2. obter o conjunto de mensagens vistas por cada nó;
3. identificar as mensagens que o outro nó ainda não possui;
4. filtrar mensagens com TTL maior que zero;
5. copiar a mensagem para o receptor, decrementando o TTL;

6. registrar a transmissão;
7. registrar entrega caso o receptor seja o destino final.

O uso da lista de mensagens vistas evita loops, pois um nó não aceita novamente uma mensagem que já recebeu anteriormente. O TTL limita a quantidade máxima de saltos de cada mensagem, evitando propagação indefinida.

6 Interface de Execução

A execução principal é feita por linha de comando. Um exemplo de execução do cenário em cadeia com roteamento epidêmico é:

```
python3 dtp2p.py simulate \  
  --contacts docs/cenarios/cadeia/contacts.txt \  
  --messages docs/cenarios/cadeia/messages.txt \  
  --router epidemic \  
  --log-level info
```

Também foi implementado suporte ao roteamento PROPHET como extensão:

```
python3 dtp2p.py simulate \  
  --contacts docs/cenarios/denso/contacts.txt \  
  --messages docs/cenarios/denso/messages.txt \  
  --router prophet
```

7 Cenários de Teste

Foram preparados três cenários principais:

- **Cenário denso:** possui muitos contatos e múltiplos caminhos alternativos;
- **Cenário esparso:** possui poucos contatos e longos intervalos sem conectividade;
- **Cenário em cadeia:** os nós se conectam em sequência, exigindo encaminhamento por intermediários.

Esses cenários permitem observar o comportamento da rede em diferentes níveis de conectividade. No cenário denso, espera-se alta taxa de entrega e menor latência. No cenário esparso, espera-se maior latência e possibilidade de mensagens não entregues. No cenário em cadeia, verifica-se diretamente o funcionamento do modelo *store-and-forward*.

8 Métricas

As métricas avaliadas foram:

- **Taxa de entrega:** razão entre mensagens entregues e mensagens criadas;
- **Latência média:** tempo médio entre criação e entrega das mensagens entregues;
- **Overhead:** razão entre o total de transmissões e o total de entregas únicas.

9 Resultados Obtidos

A Tabela 1 apresenta os resultados obtidos com o roteamento epidêmico.

Tabela 1: Resultados dos cenários com roteamento epidêmico

Cenário	Mensagens	Entregas	Taxa	Latência média	Overhead
Denso	4	4	100,00%	1,70s	3,00
Esparso	4	3	75,00%	22,33s	2,33
Cadeia	3	2	66,67%	12,75s	4,00

Os resultados confirmam o comportamento esperado. O cenário denso obteve entrega completa, pois havia maior disponibilidade de contatos e caminhos alternativos. O cenário esparso apresentou menor taxa de entrega e maior latência, evidenciando o impacto da baixa conectividade. O cenário em cadeia demonstrou o encaminhamento por nós intermediários, caracterizando o funcionamento do modelo DTN.

10 Testes

A implementação possui testes automatizados usando `unittest`. A suíte cobre parsing de contatos, parsing de mensagens, simulação por eventos, TTL, cenários principais e a extensão PROPHET.

O comando utilizado para validação foi:

```
python3 -m unittest discover -s tests -v
```

Todos os testes executados passaram com sucesso.

11 Extensão PROPHET

Além do roteamento epidêmico, foi implementada uma extensão com o roteamento PROPHET (*Probabilistic Routing Protocol using History of Encounters and Transitivity*). Esse algoritmo utiliza probabilidades de entrega baseadas no histórico de encontros entre nós, buscando reduzir transmissões desnecessárias quando comparado ao roteamento epidêmico.

No cenário denso, por exemplo, a execução com PROPHET entregou todas as mensagens, com latência média de 1,95s e overhead de 2,50, indicando redução de transmissões em relação ao roteamento epidêmico no mesmo cenário.

12 Conclusão

A atividade foi implementada com sucesso. O sistema simula uma rede P2P em ambiente DTN, sem servidor central de entrega, utilizando contatos intermitentes, buffers por nó, controle de mensagens vistas, TTL e roteamento epidêmico.

Os cenários testados demonstram a influência da conectividade na taxa de entrega, na latência e no overhead. A implementação atende aos requisitos mínimos da atividade e ainda inclui uma extensão com PROPHET para comparação com o roteamento epidêmico.